Bridge Day 7 STUDENT WORKBOOK

Ordered Validation and First-True Behavior

Learning sequence: Predict - Trace - Explain - Test - Interpret

Guided engineering evidence workbook

Name		UIN		Section	

Concrete engineering situation

The sensor-kit checkout table now uses several rules in a fixed order. A kit might have more than one issue, but the branch outcome should identify the first action the team should take. Today the focus is ordered validation: trace which condition is first true and explain why later branches are skipped.

Engineering task	Evaluate a sensor kit through an ordered checkout policy without losing evidence about skipped issues.
Computational move	Read an `if`/`elif`/`else` chain as an ordered list of checks where the first true branch wins.
Python focus	`if`, `elif`, `else`, first-true behavior, skipped branches, comparison and string-label checks.
Evidence to keep	rule order, condition result, first true branch, skipped later branch, branch outcome assigned, and rule-order risk.

Day 7 working rules

1. Python checks an `if`/`elif`/`else` chain from top to bottom.
2. The first true branch runs.
3. Later branches are skipped after the first true branch.
4. Rule order can change the branch outcome.
5. A reliable branch-decision note names the first true rule and any important skipped issue.

1. Read the ordered rules before tracing cases

recognize predict

The checkout desk now uses more than one rule. Python checks the rules in order. The first true rule assigns the branch outcome and the later rules are skipped.

```
kit_label = "S-22"
charge_percent = 76
calibration_label = "current"
borrower_initials = "MR"
damage_note = ""

if damage_note != "":
    action = "repair bench"
elif charge_percent < 80:
    action = "charge station"
elif calibration_label != "current":
    action = "calibration bench"
elif borrower_initials == "":
    action = "borrower log needed"
else:
    action = "release kit"
```

Pts.	Prompt	My evidence
1	Which rule is checked first?	
1	Which rule is checked second?	
1	Which rule is true for this kit?	
1	Which branch outcome is assigned?	

2. Trace first-true behavior

trace explain

Complete the table. Stop at the first true rule for each case.

Pts.	Damage?	Charge	Calibration	Initials	First true rule and branch outcome
2	no	76	current	MR	
2	cracked case	76	expired	MR	
2	no	91	expired	MR	
2	no	91	current	blank	

First-true reminder

If an early rule is true, Python does not continue looking for a later, also-true rule. Your evidence should name the first true rule, not just every possible problem.

3. Explain skipped branches

explain debug

A kit may have more than one problem. Ordered selection decides which problem is handled first.

```
charge_percent = 72
calibration_label = "expired"
damage_note = ""

if damage_note != "":
    action = "repair bench"
elif charge_percent < 80:
    action = "charge station"
elif calibration_label != "current":
    action = "calibration bench"
else:
    action = "release kit"
```

Pts.	Prompt	My response
2	Which branch outcome is assigned?	
2	Is the calibration rule also true for this kit? Explain.	
2	Why does the calibration branch not run?	
2	What would be dangerous about reporting only the final branch outcome without the skipped issue?	

4. Find a rule-order problem

debug test

A teammate proposes moving the release check above the damage check.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    action = "release kit"
elif damage_note != "":
    action = "repair bench"
else:
    action = "hold for review"
```

Pts.	Prompt	My response
2	What important condition is checked too late?	
2	Give a kit case that would be assigned unsafely by the proposed order.	
2	What branch outcome should that case receive?	
2	Write one sentence explaining the rule-order risk.	

5. Constrained edit of one rule

implement debug

Only edit the calibration rule below. Do not rewrite the whole decision chain.

```
elif _________________________________________________:
    action = "calibration bench"
```

Pts.	Prompt	My response
3	Complete the calibration rule.	
2	Explain why the rule should compare to `"current"`.	
2	Give one calibration label that should produce the calibration bench.	
1	Name one later branch that would be skipped if this rule is true.	

6. Transfer and support plan

transfer explain

Pts.	Ordered-validation note
4	Write a note that explains how ordered validation differs from a single `if`/`else`. Use first true and skipped branch in your explanation.
2	Circle the support plan you would use next: rule order / skipped branch / comparison / string label / written explanation. Name one next action: _____

Bridge to Day 8

Day 8 uses expected-vs-actual evidence to debug a branch decision when the selected action does not match the rule intention.

7. Lab bridge: complete the body of a provided wrapper

VIP 18.8 unscored

What stays fixed

Keep the `def` line, parameter names, and exact returned strings. You are not designing a function definition. You are completing the indented decision body that the notebook already provides. This page adds no points; the workbook score remains unchanged.

```
def badge_sheet_plan(group_count, badges_per_group):
    total_badges = group_count * badges_per_group
    if _____________________________________________:
        return "one sheet"
    elif _____________________________________________:
        return "two sheets"
    else:
        return "print batch"
```

Total badges	Expected branch outcome	First true condition	Why this is a boundary case
8	one sheet		
9	two sheets		
16	two sheets		
17	print batch		

Exact-output contract

The visible checks compare exact returned strings. Treat `"one sheet"`, `"two sheets"`, and `"print batch"` as fixed output labels, not wording to improve.

Bridge Day 7 STUDENT WORKBOOK

VIP Evidence Handoff

Learning sequence: Predict - Trace - Explain - Test - Interpret

Contract 07a04826c975 • longitudinal template 07242026_v1

Why engineers use this page

Carry comparison and branch-order evidence into the current top-level decision cells; Activity 18.14 supplies its loop.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: comparison expression, boolean operator, if elif else, abs call, counter update
Carried authoring tools: assignment, print call, numeric literal, arithmetic expression, input call, int conversion, float conversion, f string
Supplied-code exposure only: for loop, list traversal
An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.13	Compare With Tolerance	`18-13-work / 18-13-transfer / 18-13-cold`	comparison expression, f string, float conversion, input call
18.14	Count High Readings	`18-14-work / 18-14-transfer / 18-14-cold`	arithmetic expression, comparison expression, counter update, f string, if elif else
18.15	Lamination Fee	`18-15-work / 18-15-transfer / 18-15-cold`	comparison expression, f string, if elif else, input call, int conversion
18.16	Temperature Status	`18-16-work / 18-16-transfer / 18-16-cold`	comparison expression, f string, float conversion, if elif else, input call
18.17	Pickup Window	`18-17-work / 18-17-transfer / 18-17-cold`	boolean operator, comparison expression, f string, if elif else, input call, int conversion

Readiness evidence

1. Identify which activity supplies a loop and state exactly what decision you are responsible for writing inside it.

2. For one of Activities 18.13, 18.15, 18.16, or 18.17, name an equality boundary that must receive its own test.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	
Observed Result	
Revision Reason	
Engineering Meaning	
Units Or Not Applicable	
Assumption	
Model Limit	
Next Test	
Help Trigger	
Minutes Used	

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Generated from the canonical VIP construct and evidence contract; workbook planning and executable notebook work remain distinct.